

🔖 Bookmark

Домашнее задание по теме: Использование Ansible

Подготовка к выполнению

1. Подготовьте в Yandex Cloud три хоста: для `clickhouse`, для `vector` и для `lighthouse`.
2. Репозиторий LightHouse находится [по ссылке](#).

Основная часть

1. Допишите `playbook`: нужно сделать ещё один `play`, который устанавливает и настраивает `LightHouse`.
2. При создании `tasks` рекомендую использовать модули: `get_url`, `template`, `yum`, `apt`.
3. Tasks должны: скачать статику `LightHouse`, установить `Nginx` или любой другой веб-сервер, настроить его конфиг для открытия `LightHouse`, запустить веб-сервер.
4. Подготовьте свой `inventory-файл` `prod.yml`.
5. Запустите `ansible-lint site.yml` и исправьте ошибки, если они есть.
6. Попробуйте запустить `playbook` на этом окружении с флагом `--check`.
7. Запустите `playbook` на `prod.yml` окружении с флагом `--diff`. Убедитесь, что изменения на системе произведены.

8. Повторно запустите playbook с флагом `--diff` и убедитесь, что `playbook` идемпотентен.
9. Подготовьте README.md-файл по своему `playbook`. В нём должно быть описано: что делает `playbook`, какие у него есть параметры и теги.
10. Готовый `playbook` выложите в свой репозиторий, поставьте тег `08-ansible-03-yandex` на фиксирующий коммит, в ответ предоставьте ссылку на него.

Развернул в Yandex Cloud 3 VM скриптом с настройкой

```
1  #!/bin/bash
2  set -euo pipefail
3
4  ZONE="ru-central1-a"
5  SUBNET="default-ru-central1-a"
6  IMAGE_FOLDER="standard-images"
7  IMAGE_FAMILY="almalinux-9"
8  CLOUD_INIT_FILE="cloud-init.yaml"
9
10 echo "Create clickhouse-01"
11 yc compute instance create \
12     --name clickhouse-01 \
13     --hostname clickhouse-01 \
14     --zone "$ZONE" \
15     --network-interface subnet-name="$SUBNET",nat-ip-version=ipv4 \
16     --create-boot-disk image-folder-id="$IMAGE_FOLDER",image-
family="$IMAGE_FAMILY",size=20 \
```

```
17  --memory 4 \
18  --cores 2 \
19  --metadata-from-file user-data="$CLOUD_INIT_FILE"
20
21  echo "Create vector-01"
22  yc compute instance create \
23    --name vector-01 \
24    --hostname vector-01 \
25    --zone "$ZONE" \
26    --network-interface subnet-name="$SUBNET",nat-ip-version=ipv4 \
27    --create-boot-disk image-folder-id="$IMAGE_FOLDER",image-
family="$IMAGE_FAMILY",size=20 \
28    --memory 4 \
29    --cores 2 \
30    --metadata-from-file user-data="$CLOUD_INIT_FILE"
31
32  echo "Create lighthouse-01"
33  yc compute instance create \
34    --name lighthouse-01 \
35    --hostname lighthouse-01 \
36    --zone "$ZONE" \
37    --network-interface subnet-name="$SUBNET",nat-ip-version=ipv4 \
38    --create-boot-disk image-folder-id="$IMAGE_FOLDER",image-
family="$IMAGE_FAMILY",size=20 \
39    --memory 2 \
```

```
40  --cores 2 \  
41  --metadata-from-file user-data="$CLOUD_INIT_FILE"  
42  
43  echo  
44  echo "Instances list:"  
45  yc compute instance list
```

```
1  chmod +x create-VM.sh
```

Создал файл `cloud-init.yaml` для автоматического выполнения при первом запуске VM

```
1  #cloud-config  
2  users:  
3    - name: kva  
4      groups: wheel  
5      shell: /bin/bash  
6      sudo: ['ALL=(ALL) NOPASSWD:ALL']  
7      ssh_authorized_keys:  
8        - ssh-ed25519  
9        AAAAC3NzaC1lZDI1NTE5AAAAICGCxz6Ttx4jaiMvpdANwl4va+6RTIWd+Vd8yLDNu1xD vboxuser@KVA  
10  
11  package_update: true  
12  package_upgrade: false  
13  packages:  
14    - python3
```

Комментарий к скрипту:

`set -euo pipefail` - набор настроек Bash, в котором: `-e` завершает скрипт при ошибке команды, `-u` запрещает использовать необъявленные переменные, а `set -o pipefail` делает так, чтобы ошибка в любой команде конвейера считалась ошибкой всего конвейера.

Запустил. Машины развернулись

```
vboxuser@KVA:~/ansible/hw-3$ ./create-VM.sh
```

Instances list:

ID	NAME	ZONE ID	STATUS	EXTERNAL IP	INTERNAL IP
fhm364ftam5e2o09p2pm	vector-01	ru-central1-a	RUNNING	93.77.190.67	10.128.0.17
fhm dtelsbg5g5u8q4sf3	lighthouse-01	ru-central1-a	RUNNING	51.250.90.29	10.128.0.36
fhm luhu9vj423n14ngrf	clickhouse-01	ru-central1-a	RUNNING	93.77.182.122	10.128.0.19

Записал параметры: `ip`, `ansible_user: kva` и `ansible_python_interpreter: /usr/bin/python3`, а также хост `lighthouse` в файл `/playbook/inventory/prod.yml`

```

1  ---
2  all:
3      vars:
4          ansible_user: kva
5          ansible_python_interpreter: /usr/bin/python3
6
7  clickhouse:
```

```
8     hosts:
9         clickhouse-01:
10             ansible_host: 93.77.182.122
11
12     vector:
13         hosts:
14             vector-01:
15                 ansible_host: 93.77.190.67
16
17     lighthouse:
18         hosts:
19             lighthouse-01:
20                 ansible_host: 51.250.90.29
```

Добавил сюда пользователя ansible - по имени пользователя для VM из файла `cloud.init` и явно указал интерпретатор `python`

Создал файл `playbook/group_vars/lighthouse/vars.yml` со следующим содержимым:

```
1 ---
2 lighthouse_url: "https://github.com/VKCOM/lighthouse/archive/refs/heads/master.zip"
3 lighthouse_dir: "/usr/share/nginx/html/lighthouse"
4 lighthouse_nginx_config: "/etc/nginx/conf.d/lighthouse.conf"
```

- `lighthouse_url` - откуда скачивать архив со статикой (репозиторий заархивирован и имеет статус `read only`);
- `lighthouse_dir` - куда распаковывать сайт;
- `lighthouse_nginx_config` - куда положить конфиг Nginx.

Создал файл `playbook/templates/lighthouse_nginx.conf.j2`

```
1  server {
2      listen 80;
3      server_name _;
4
5      root {{ lighthouse_dir }};
6      index index.html;
7
8      location / {
9          try_files $uri $uri/ /index.html;
10     }
11 }
```

- `root {{ lighthouse_dir }}` говорит Nginx, откуда отдавать статические файлы интерфейса `lighthouse`.
- `index index.html` задает стартовую страницу,
- `try_files` помогает корректно обслуживать маршруты фронта, если интерфейс обращается к путям внутри приложения, а не только к прямым файлам.

Добавил третий `play` в `playbook` `site.yml` блок по установке `lighthouse` В итоге он приобрел вид:

```
1  ---
2  - name: Install Clickhouse
3    hosts: clickhouse
4    handlers:
5      - name: Start clickhouse service
6        become: true
7        ansible.builtin.service:
8          name: clickhouse-server
9          state: restarted
10   tasks:
11     - name: Get clickhouse distrib
12       block:
13         - name: Get clickhouse distrib
14           ansible.builtin.get_url:
15             url: "https://packages.clickhouse.com/rpm/stable/{{ item }}-{{
clickhouse_version }}.noarch.rpm"
16             dest: "./{{ item }}-{{ clickhouse_version }}.rpm"
17             mode: "0644"
18             with_items: "{{ clickhouse_packages }}"
19         rescue:
20           - name: Get clickhouse distrib
21             ansible.builtin.get_url:
```



```
22         url: "https://packages.clickhouse.com/rpm/stable/clickhouse-common-static-
{{ clickhouse_version }}.x86_64.rpm"
23         dest: "./clickhouse-common-static-{{ clickhouse_version }}.rpm"
24         mode: "0644"
25     when: not ansible_check_mode
26
27 - name: Install clickhouse packages
28   become: true
29   ansible.builtin.dnf:
30     name:
31     - clickhouse-common-static-{{ clickhouse_version }}.rpm
32     - clickhouse-client-{{ clickhouse_version }}.rpm
33     - clickhouse-server-{{ clickhouse_version }}.rpm
34     disable_gpg_check: true
35   notify: Start clickhouse service
36   when: not ansible_check_mode
37
38 - name: Flush handlers
39   ansible.builtin.meta: flush_handlers
40   when: not ansible_check_mode
41
42 - name: Create database
43   ansible.builtin.command: "clickhouse-client -q 'create database logs;'"
44   register: create_db
45   failed_when: create_db.rc != 0 and create_db.rc != 82
```

```
46     changed_when: create_db.rc == 0
47     when: not ansible_check_mode
48
49 - name: Install Vector
50   hosts: vector
51   handlers:
52     - name: Validate vector config
53       ansible.builtin.command:
54         cmd: "{{ vector_install_dir }}/bin/vector validate {{ vector_install_dir }}
55             }}/config/vector.yml"
56         changed_when: false
57
58   tasks:
59     - name: Create vector install directory
60       ansible.builtin.file:
61         path: "{{ vector_install_dir }}"
62         state: directory
63         mode: "0755"
64       when: not ansible_check_mode
65
66     - name: Get Vector distrib
67       ansible.builtin.get_url:
68         url: "https://github.com/vectordev/vector/releases/download/v{{
69             vector_version }}/vector-{{ vector_version }}-x86_64-unknown-linux-musl.tar.gz"
70         dest: "/tmp/vector-{{ vector_version }}.tar.gz"
```

```
69     mode: "0644"
70     when: not ansible_check_mode
71
72 - name: Unarchive Vector
73   ansible.builtin.unarchive:
74     src: "/tmp/vector-{{ vector_version }}.tar.gz"
75     dest: "{{ vector_install_dir }}"
76     remote_src: true
77     extra_opts:
78       - "--strip-components=2"
79     creates: "{{ vector_install_dir }}/bin/vector"
80   when: not ansible_check_mode
81
82 - name: Create vector config directory
83   ansible.builtin.file:
84     path: "{{ vector_install_dir }}/config"
85     state: directory
86     mode: "0755"
87   when: not ansible_check_mode
88
89 - name: Deploy vector config
90   ansible.builtin.template:
91     src: vector.yml.j2
92     dest: "{{ vector_install_dir }}/config/vector.yml"
93     mode: "0644"
```

```
94     notify: Validate vector config
95     when: not ansible_check_mode
96
97 ##### добавлен блок по установке и настройке Lighthouse
98
99 - name: Install Lighthouse
100   hosts: lighthouse
101   become: true
102
103   handlers:
104     - name: Restart nginx
105       ansible.builtin.service:
106         name: nginx
107         state: restarted
108
109   tasks:
110     - name: Install Nginx and unzip
111       ansible.builtin.dnf:
112         name:
113           - nginx
114           - unzip
115         state: present
116       when: not ansible_check_mode
117
118     - name: Create lighthouse directory
```

```
119     ansible.builtin.file:
120         path: "{{ lighthouse_dir }}"
121         state: directory
122         mode: "0755"
123     when: not ansible_check_mode
124
125 - name: Create temporary directory for Lighthouse archive
126     ansible.builtin.file:
127         path: /tmp/lighthouse
128         state: directory
129         mode: "0755"
130     when: not ansible_check_mode
131
132 - name: Download Lighthouse archive
133     ansible.builtin.get_url:
134         url: "{{ lighthouse_url }}"
135         dest: "/tmp/lighthouse.zip"
136         mode: "0644"
137     when: not ansible_check_mode
138
139 - name: Unarchive Lighthouse
140     ansible.builtin.unarchive:
141         src: "/tmp/lighthouse.zip"
142         dest: "/tmp/lighthouse"
143         remote_src: true
```

```
144         creates: "/tmp/lighthouse/lighthouse-master/index.html"
145     when: not ansible_check_mode
146
147 - name: Copy Lighthouse files
148   ansible.builtin.copy:
149     src: "/tmp/lighthouse/lighthouse-master/"
150     dest: "{{ lighthouse_dir }}"
151     remote_src: true
152     mode: preserve
153   notify: Restart nginx
154   when: not ansible_check_mode
155
156 - name: Deploy Nginx config
157   ansible.builtin.template:
158     src: lighthouse_nginx.conf.j2
159     dest: "{{ lighthouse_nginx_config }}"
160     mode: "0644"
161   notify: Restart nginx
162   when: not ansible_check_mode
163
164 - name: Start Nginx service
165   ansible.builtin.service:
166     name: nginx
167     state: started
168     enabled: true
```

```
169     when: not ansible_check_mode
170
```

- `become: true` : установка пакетов, запись в `/etc/nginx` и `/usr/share/nginx/html` требуют root-прав. Поэтому для всего `play` сразу включены соответствующие права.
- `unzip` потому что скачивается ZIP-архив репозитория `LightHouse`, а не `git clone`. Для распаковки ZIP на удаленном хосте системе нужен инструмент распаковки, и на RPM-системах это обычно пакет `unzip`; модуль `unarchive` работает с архивом на `managed node` при `remote_src: true`. Скачиваем и распаковываем во временную папку, потом `ansible.builtin.copy` переносим файлы в рабочую директорию `lighthouse`
- `creates: "{{ lighthouse_dir }}/index.html"` для того, чтобы не разархивировать повторно архив, если он ранее уже создан.

Делаю `ansible-lint site.yml` проверку кода

```
vboxuser@KVA:~/ansible/hw-3/playbook$ ansible-lint site.yml
Passed: 0 failure(s), 0 warning(s) in 1 files processed of 1 encountered. Last profile that met the validation criteria was 'production'.
vboxuser@KVA:~/ansible/hw-3/playbook$
```

Предварительно, код верен и устраивает ansible

Дальнейшая проверка кода: `ansible-playbook -i inventory/prod.yml site.yml --check`

На посыпавшиеся ошибки, для корректности добавил в конце каждого `task` строку `when: not ansible_check_mode` которая разрешает не запускать полный сценарий скачивания и всего остального в `check` режиме. На прошлом задании применял `check_mode: false`, поскольку работа велась локально и для `vector` файлы скачивались в необходимые директории. С учетом сценария `dry-run` уместнее применять `when: not ansible_check_mode` для всех блоков

```
vboxuser@KVA:~/ansible/hw-3/playbook$ ansible-playbook -i inventory/prod.yml site.yml --check
```

```
PLAY [Install Clickhouse] *****
```

```
TASK [Gathering Facts] *****
```

```
ok: [clickhouse-01]
```

```
TASK [Get clickhouse distrib] *****
```

```
skipping: [clickhouse-01] => (item=clickhouse-client)
```

```
skipping: [clickhouse-01] => (item=clickhouse-server)
```

```
skipping: [clickhouse-01] => (item=clickhouse-common-static)
```

```
skipping: [clickhouse-01]
```

```
TASK [Install clickhouse packages] *****
```

```
skipping: [clickhouse-01]
```

```
TASK [Flush handlers] *****
```

```
[WARNING]: Callback dispatch 'v2_runner_on_skipped' failed for plugin 'default': CallbackModule.v2_runner_on_skipped() takes 2 positional arguments but 4 were given
```

```
TASK [Create database] *****
```

```
skipping: [clickhouse-01]
```

```
PLAY [Install Vector] *****
```

```
TASK [Gathering Facts] *****
```

```
ok: [vector-01]
```

```
TASK [Create vector install directory] *****
```

```
skipping: [vector-01]
```

```
TASK [Get Vector distrib] *****
```

```
skipping: [vector-01]
```

```
TASK [Unarchive Vector] *****
```

```
skipping: [vector-01]
```

```
TASK [Create vector config directory] *****
```

```
skipping: [vector-01]
```

```
TASK [Deploy vector config] *****
```

```
skipping: [vector-01]
```

```
PLAY [Install Lighthouse] *****
```

```
TASK [Gathering Facts] *****
```

```
ok: [lighthouse-01]
```



```

TASK [Install Nginx and unzip] *****
skipping: [lighthouse-01]

TASK [Create lighthouse directory] *****
skipping: [lighthouse-01]

TASK [Create temporary directory for Lighthouse archive] *****
skipping: [lighthouse-01]

TASK [Download Lighthouse archive] *****
skipping: [lighthouse-01]

TASK [Unarchive Lighthouse] *****
skipping: [lighthouse-01]

TASK [Copy Lighthouse files] *****
skipping: [lighthouse-01]

TASK [Deploy Nginx config] *****
skipping: [lighthouse-01]

TASK [Start Nginx service] *****
skipping: [lighthouse-01]

PLAY RECAP *****
clickhouse-01      : ok=1    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
lighthouse-01     : ok=1    changed=0    unreachable=0    failed=0    skipped=8    rescued=0    ignored=0
vector-01          : ok=1    changed=0    unreachable=0    failed=0    skipped=5    rescued=0    ignored=0

```

Запустил `ansible-playbook -i inventory/prod.yml site.yml --diff`

на этапе установки `clickhouse` упало с двумя ошибками:

Первая ошибка на `clickhouse-common-static` с `404` - она специально обрабатывается через `rescue`: для `common-static` нужен `x86_64`, а не `noarch`. Фатально упало на установке: `dnf` отказался ставить локальный RPM из-за отсутствующей или неподходящей подписи. Ansible-модуль `dnf` действительно имеет параметр `disable_gpg_check`, который влияет на установку пакетов в состоянии `present/latest`. В задаче `Install clickhouse packages` добавил параметр: `disable_gpg_check: true` после команд на установку пакетов. Это локальное учебное решение, вообще документация ClickHouse рекомендует для RPM-based систем подключать официальный репозиторий через `yum-config-manager --add-repo https://packages.clickhouse.com/rpm/clickhouse.repo`, а не тащить локальные RPM вручную (<https://clickhouse.com/docs/install/redhat>).

После первого прогона с `diff` все развернулось без ошибок (за исключением блока с путями к `clickhouse` о котором написал выше.)

```
vboxuser@KVA:~/ansible/hw-3/playbook$ ansible-playbook -i inventory/prod.yml site.yml --diff
```

```
+ listen 80;
+ server_name _;
+
+ root /usr/share/nginx/html/lighthouse;
+ index index.html;
+
+ location / {
+     try_files $uri $uri/ /index.html;
+ }
+}
\ No newline at end of file
```

```
changed: [lighthouse-01]
```

```
TASK [Start Nginx service]
```

```
changed: [lighthouse-01]
```

```
RUNNING HANDLER [Restart nginx]
```

```
changed: [lighthouse-01]
```

```
PLAY RECAP
```

clickhouse-01	: ok=4	changed=0	unreachable=0	failed=0	skipped=0	rescued=1	ignored=0
lighthouse-01	: ok=10	changed=9	unreachable=0	failed=0	skipped=0	rescued=0	ignored=0
vector-01	: ok=5	changed=0	unreachable=0	failed=0	skipped=1	rescued=0	ignored=0

Второй прогон с `diff` ничего не менял в уже ранее проиленных шагах.

```
PLAY RECAP
```

clickhouse-01	: ok=4	changed=0	unreachable=0	failed=0	skipped=0	rescued=1	ignored=0
lighthouse-01	: ok=8	changed=0	unreachable=0	failed=0	skipped=1	rescued=0	ignored=0
vector-01	: ok=5	changed=0	unreachable=0	failed=0	skipped=1	rescued=0	ignored=0